



Diversity Guided Evolutionary Mining of Hierarchical Process Models

Thomas Molka
HANA Cloud Computing,
Systems Engineering, SAP
Belfast, UK
thomasmolka@gmail.com

David Redlich
HANA Cloud Computing,
Systems Engineering, SAP
Belfast, UK
d.redlich@gmx.de

Marc Drobek
HANA Cloud Computing,
Systems Engineering, SAP
Belfast, UK
marcdrobek@gmx.de

Xiao-Jun Zeng
The University of Manchester
Manchester, UK
x.zeng@manchester.ac.uk

Wasif Gilani
HANA Cloud Computing,
Systems Engineering, SAP
Belfast, UK
wasif.gilani@sap.com

ABSTRACT

Easy-to-understand and up-to-date models of business processes are important for enterprises, as they aim to describe how work is executed in reality and provide a starting point for process analysis and optimization. With an increasing amount of event data logged by information systems today, the automatic discovery of process models from process logs has become possible. Whereas most existing techniques focus on the discovery of well-formalized models (e.g. Petri nets) which are popular among researchers, business analysts prefer business domain-specific models (such as Business Process Model Notation, BPMN) which are not well formally specified. We present and evaluate an approach for discovering the latter type of process models by formally specifying a hierarchical view on business process models and applying an evolution strategy on it. The evolution strategy efficiently finds process models which best represent a given event log by using fast methods for process model conformance checking, and is partly guided by the diversity of the process model population. The approach contributes to the field of evolutionary algorithms by showing that they can be successfully applied in the real-world use case of process discovery, and contributes to the process discovery domain by providing a promising alternative to existing methods.

Keywords

business process management; process mining; evolutionary algorithms; population diversity

1. INTRODUCTION

In today's enterprises, the work carried out is organized in business processes. These processes define a collection of tasks along with their order of execution needed to achieve a certain business goal. Documentation on these processes in the shape of process models

is essential to analyzing and optimizing the process, as well as for process stakeholders to make informed decisions. In most cases, however, there are no such models available. Often, the available models are created manually and become outdated when the underlying process is adapted or optimized, or simply do not reflect the process as executed in reality.

At the same time, processes are increasingly supported by information systems which, as a by-product, create event logs documenting the execution of the tasks along with additional information such as the time of execution or the resources involved. These event logs can be used to mine (i.e. automatically extract) process models, models describing the interaction of resources, or even simulation models used for process performance predictions [13, 14]. While most process mining techniques focus on the extraction of well-formalized models such as Petri nets, these models tend to get complex and difficult to understand, especially in case of larger processes [12]. Business process stakeholders prefer less formal models from the business process domain, such as BPMN [10], which are typically used to document processes when drawn manually by business process analysts.

We propose an approach which uses an evolution strategy to mine business process domain models that allow the consumer to abstract from complex behaviour by viewing the process at a desired level of detail. The approach can deal with noise in event logs and allows the user to configure the set of business process constructs permitted in the mined models.

In the remainder of this paper we formally define the model we use for representing business processes (Section 2), define a set of operations for modifying these kind of models (Section 3), and explain the evolution strategy used to mine them (Section 4). We conclude by evaluating the approach (Section 5), setting it into the context of related work (Section 6), and summarizing our results (Section 7).

2. PROCESS MODEL

Our genotype for the optimization problem is constituted by two representations of business process. Firstly, we use a process model which consists of basic business process control-flow elements as described in Section 2.1. For the implementation of this model we take a definition provided in [11] which fulfils our requirements regarding well-structuredness of process models, and for which process conformance measures are already defined (measures for determining how well a process model reflects the real execution of

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

GECCO '15, July 11 - 15, 2015, Madrid, Spain

© 2015 Copyright held by the owner/author(s). Publication rights licensed to ACM. ISBN 978-1-4503-3472-3/15/07...\$15.00

DOI: <http://dx.doi.org/10.1145/2739480.2754765>

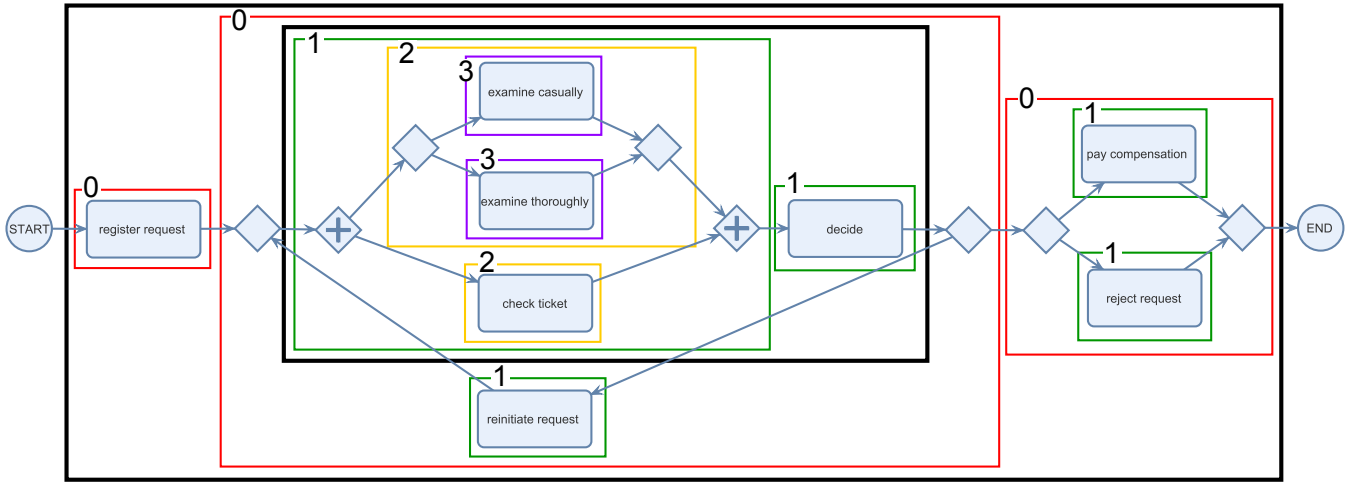


Figure 1: Example of a business process model and its hierarchical view.

the process as documented in an event log). Secondly, the genotype is expressed as a hierarchical view which we define as an abstraction layer on top of the control-flow (Section 2.2). This hierarchical view provides the user with easy-to-understand models and allows the evolution strategy to perform efficient operations.

2.1 Control-Flow

The set of business process elements used in our models (also supported by the most common business process modelling languages BPMN and EPC) consists of: event nodes marking the *start* and *end* of the process, *activities* which represent process steps in which work is carried out, *decisions* representing a split of a branch into exclusive choice branches, *merges* to connect exclusive branches again, *forks* for splitting a branch into parallel branches, and *joins* to synchronize parallel branches.

In the following, we use a definition given in [11] as formal description of the control-flow model. There, a business process model is defined as a tuple $BP = (B, C)$ containing a set of business process elements $B = B_A \cup B_D \cup B_M \cup B_F \cup B_J \cup \{s\} \cup \{e\}$, and a connection relation $C \subseteq B \times B$. The set of business process elements consists of activities B_A , decisions B_D , merges B_M , forks B_F , joins B_J , exactly one start event s , and exactly one end event e . The connection relation C describes which business process element follows another process element (being one of its “targets”) or is followed by another process element (being one of its “sources”) [11]:

- $C \subseteq \{(a, b) \in B \times B \mid a \neq b \wedge a \neq e \wedge b \neq s\}$ (an element can’t be its own source or target, the end element has no target, and the start element has no source),
- $\forall a \in B_A \cup B_M \cup B_J \cup \{s\} : |\{(a, b) \in C \mid b \in B\}| = 1$ (activities, merges, joins, and the start element have exactly one target),
- $\forall a \in B_A \cup B_D \cup B_F \cup \{e\} : |\{(b, a) \in C \mid b \in B\}| = 1$ (activities, decisions, forks, and the end element have exactly one source),
- $\forall a \in B_M \cup B_J : |\{(b, a) \in C \mid b \in B\}| \geq 1$ (merges and joins have at least one source element),
- $\forall a \in B_D \cup B_F : |\{(a, b) \in C \mid b \in B\}| \geq 1$ (decisions and forks have at least one target element), and
- $\forall a \in B : \exists (s, a) \in C^+ \wedge \exists (a, e) \in C^+$ (all elements in B of the process (B, C) are on a path from s to e (C^+ being the transitive closure of C))

2.2 Hierarchical Model

In order to provide a better structure for operations on the genotype, i.e. the process model, carried out by the evolutionary algorithm, we create a hierarchical view on process models, consisting of two types of elements: *atomic graphs* $\mathcal{A}$ and *sequences of atomic graphs* $\mathcal{S}$. Atomic graphs are constructs representing a part of the control-flow model. This part can either be a single activity, or a block structure (exclusive, parallel, or loop) which consists of two branches (branches are sequences of atomic graphs). Figure 1 shows an example of the control-flow model of a process and a hierarchical view on it (marked by additional frames around parts of the control-flow). The control-flow contains activities in the shape of rounded rectangles, gateways in the shape of diamonds (empty diamonds denote decisions and merges, diamonds with a “+” denote forks and joins), and the start and end node (circles). The black rectangles with thick stroke indicate sequences of atomic graphs with a length of two or more. All other rectangles frame atomic graphs. The number on the top left of each rectangle represents the hierarchy level of the atomic graph. The main process sequence contains three atomic graphs (on hierarchy level 0): a simple activity, a loop construct, and an exclusive choice construct. The loop construct has a parallel construct in one of its branches, which in turn contains an exclusive construct, extending the hierarchy level to 3.

In the following we provide the formal definition for hierarchical views. Let $B = B_A \cup B_D \cup B_M \cup B_F \cup B_J \cup \{s\} \cup \{e\}$ be a set of business process control flow elements (cf. Section 2.1). Then a hierarchical view $h = (\mathcal{A}, \mathcal{S}, B)$ on them is defined by:

- $\mathcal{A} = \mathcal{A}_A \cup \mathcal{A}_E \cup \mathcal{A}_P \cup \mathcal{A}_L$ (atomic graphs can be atomic activities, exclusive constructs, parallel constructs, or loops)
- $\mathcal{S} = \mathcal{S}_B \cup \{s_p\}$ (the set of sequences consists of branch sequences and one “root” process sequence s_p)
- $\forall s \in \mathcal{S} \exists n \in \mathbb{N} \exists A \subseteq \mathcal{A} : s \in \{0 \dots n\} \rightarrow A$ (sequences are functions assigning each index an atomic graph)
- $\forall (f, l, s_1, s_2) \in \mathcal{A} : f, l \in B \wedge s_1, s_2 \in \mathcal{S}_B \cup \{\$ \}$ (all atomic graphs have a first and a last control flow element, and two branch sequences)
- $\forall (a, a, \$, \$) \in \mathcal{A}_A : a \in B_A$ (atomic activities contain only one activity and no branch sequences, the \$ sign shall express the lack of a branch)

- $\forall e \in \mathcal{A}_E \exists d \in B_D \exists m \in B_M \exists s_1, s_2 \in \mathcal{S} : e = (d, m, s_1, s_2)$ (exclusive constructs have a decision as first element, a merge as last element, and two branch sequences)
- $\forall l \in \mathcal{A}_L \exists d \in B_D \exists m \in B_M \exists s_1, s_2 \in \mathcal{S} : l = (m, d, s_1, s_2)$ (loop constructs have a merge as first element, a decision as last element, and two branch sequences)
- $\forall p \in \mathcal{A}_P \exists f \in B_F \exists j \in B_J \exists s_1, s_2 \in \mathcal{S} : p = (f, j, s_1, s_2)$ (parallel constructs have a fork as first element, a join as last element, and two branch sequences)
- $\forall a \in \mathcal{A} \exists_{=1} s \in \mathcal{S} \exists_{=1} n < |s| : (n, a) \in s$ (every atomic graph occurs exactly once in exactly one sequence of atomic graphs, i.e. every atomic graph is unique)
- $\forall s \in \mathcal{S}_B \exists_{=1} (f, l, s_1, s_2) \in \mathcal{A} : s = s_1 \vee s = s_2$ (every branch sequence is a branch of exactly one atomic graph, i.e. every branch sequence is contained in exactly one atomic graph)

3. PROCESS MODEL OPERATORS

In this section we define a set of operators for modifying a hierarchical model. These operators are used by the evolutionary algorithm described in Section 4. The operators we use are *move* ($\rightsquigarrow$) for moving parts of the process model to a different position in the model; *complexify* ($\oplus$, $\otimes$, $\odot$) for introducing complex constructs such as parallel, exclusive or loop constructs; and *simplify* ($\circ$) for resolving complex constructs into a sequential shape. As we know all activities that are to be part of the model (by pre-processing the event log) we do not need to remove or insert activities in the model, but only the three above-mentioned operators for changing the structure of how the activities are connected.

The $\rightsquigarrow$ operator is used to move a subsequence of atomic graphs to a different part of the model (i.e. into another sequence of atomic graphs, or into a different position within the same sequence). The destinations of each move are limited to sequences which are no children of any of the graphs that are being moved (informally: an atomic graph cannot be moved into itself). Formally, in a hierarchical model $(\mathcal{A}, \mathcal{S}, B)$, the set of sequences $S \subseteq \mathcal{S}$ a set of atomic graphs $A \subseteq \mathcal{A}$ can be moved to, is defined as:

$$S = \mathcal{S} \setminus \{s \mid (a, s) \in P^+ \wedge a \in A \wedge s \in \mathcal{S}_B\}$$

where P^+ is the transitive closure of the combined parent relationship between atomic graphs and sequences:

$$P = \{(a, b) \mid (a = \text{parentOfSequence}(b) \vee a = \text{parentOfGraph}(b))\}$$

where the function returning the parent atomic graph for a sequence of atomic graphs is defined as

$$\begin{aligned} \text{parentOfSequence} : \mathcal{S}_B &\rightarrow \mathcal{A} \\ s &\mapsto (f, l, s_1, s_2) \mid s = s_1 \vee s = s_2 \end{aligned}$$

and the function returning the parent sequence for an atomic graph is defined as

$$\begin{aligned} \text{parentOfGraph} : \mathcal{A} &\rightarrow \mathcal{S} \\ a &\mapsto s \mid \exists (i, a) \in s \end{aligned}$$

Note that P contains only direct parent relations between atomic graphs and their parent sequence, and between branch sequences and their parent atomic graph, i.e. $P \subseteq (\mathcal{A} \times \mathcal{S}) \cup (\mathcal{S} \times \mathcal{A})$. In contrast, P^+ additionally contains the indirect (transitive) relations on the set of atomic graphs and sequences, i.e. $P^+ \subseteq (\mathcal{A} \cup \mathcal{S}) \times (\mathcal{A} \cup \mathcal{S})$.

In order to shorten the definition of the $\rightsquigarrow$ operator, we first define two functions on sequences of atomic graphs. The “remove” function $\text{rmv}(s, k, l)$ returns a sequence of atomic graphs representing sequence s without the atomic graphs at indices k to l (the atomic graph at index $l + 1$ is then at index k etc.):

$$\begin{aligned} \text{rmv} : (\mathcal{S}, \mathbb{N}, \mathbb{N}) &\rightarrow \mathcal{S} \\ \text{rmv}(s, k, l) &= (s \setminus \{(i, a) \in s \mid i \geq k\}) \\ &\quad \cup \{(i + k - l - 1, a) \mid (i, a) \in s \wedge i > l\} \end{aligned}$$

The “insert” function $\text{ins}(s_1, k, l, s_2, m)$ returns a sequence containing all the atomic graphs of sequence s_2 with some additional atomic graphs from sequence s_1 (the ones at indices k to l in s_1) inserted at index m (the atomic graphs after index m being moved back by $l - k + 1$):

$$\begin{aligned} \text{ins} : (\mathcal{S}, \mathbb{N}, \mathbb{N}, \mathcal{S}, \mathbb{N}) &\rightarrow \mathcal{S} \\ \text{ins}(s_1, k, l, s_2, m) &= (s_2 \setminus \{(i, a) \in s_2 \mid i \geq m\}) \\ &\quad \cup \{(i + l - k + 1, a) \mid (i, a) \in s_2 \wedge i \geq m\} \\ &\quad \cup \{(i + k, a) \mid (i + k, a) \in s_1 \wedge i \leq l - k\} \end{aligned}$$

For instance, $\text{ins}(s_1, 0, \infty, s_2, 4)$ would return a sequence where all atomic graphs of s_1 are inserted at index 4 in s_2 .

Using the two functions rmv and ins , we define the move operation as follows ($\mathfrak{H}$ being the domain of hierarchical models):

$$\begin{aligned} \rightsquigarrow : (\mathfrak{H}, \mathcal{S}, \mathbb{N}, \mathbb{N}, \mathcal{S}, \mathbb{N}) &\rightarrow \mathfrak{H} \\ \rightsquigarrow ((\mathcal{A}, \mathcal{S}, B), s_1, k, l, s_2, m) &= (\mathcal{A}, \mathcal{S}', B) \end{aligned}$$

where $\mathcal{S}' = (\mathcal{S} \setminus \{s_1, s_2\}) \cup \{\text{rmv}(s_1, k, l), \text{ins}(s_1, k, l, s_2, m)\}$.

When the operators $\oplus$, $\otimes$, or $\odot$ are applied on a part of a sequence of atomic graphs, this part is made more complex as it is used to create a parallel, exclusive or loop construct. This part of the sequence is split into two subparts, each of which will constitute a branch in the newly created construct. These operations modify the set $\mathcal{A}$ by adding a new atomic graph, modify the set $\mathcal{S}$ by adding two new sequences (and modifying the original sequence which contained the part), and add two new gateways to the set of control-flow elements B :

$$\begin{aligned} \text{com} : (\mathfrak{H}, \mathcal{S}, \mathbb{N}, \mathbb{N}, \mathbb{N}) &\rightarrow \mathfrak{H} \\ \text{com}((\mathcal{A}, \mathcal{S}, B), s_1, k, l, m) &= (\mathcal{A} \cup \{a\}, (\mathcal{S} \setminus \{s_1\}) \cup \{s'_1, s_2, s_3\}, \\ &\quad B \cup \{a, b\}) \end{aligned}$$

where

$$\begin{aligned} a &= (a, b, s_2, s_3) \wedge \\ s_2 &= \{(i, b) \mid k \leq i \leq l\} \wedge \\ s_3 &= \{(i, b) \mid l < i \leq m\} \wedge \\ s'_1 &= \text{ins}(\{(0, a)\}, 0, 0, \text{rmv}(s_1, k, m), k) \end{aligned}$$

The types of the two business process control-flow elements a and b introduced into the model depend on the specific operation which is applied. In case of $\oplus$ a fork and join are created so that $a \in B_F$ and $b \in B_J$. When $\otimes$ is applied a decision and a merge are created so that $a \in B_D$ and $b \in B_M$, and in case of the loop operator $\odot$ a decision and a merge are created so that $a \in B_M$ and $b \in B_D$.

We use the simplify operator $\circ$ in order to reduce model complexity by resolving complex constructs into a sequential shape. This means that an atomic graph in shape of a parallel, exclusive, or a loop construct is removed from the set $\mathcal{A}$, its two branches are removed from the set $\mathcal{S}$ and their contents are inserted into the parent sequence of the atomic graph (at the position the atomic graph

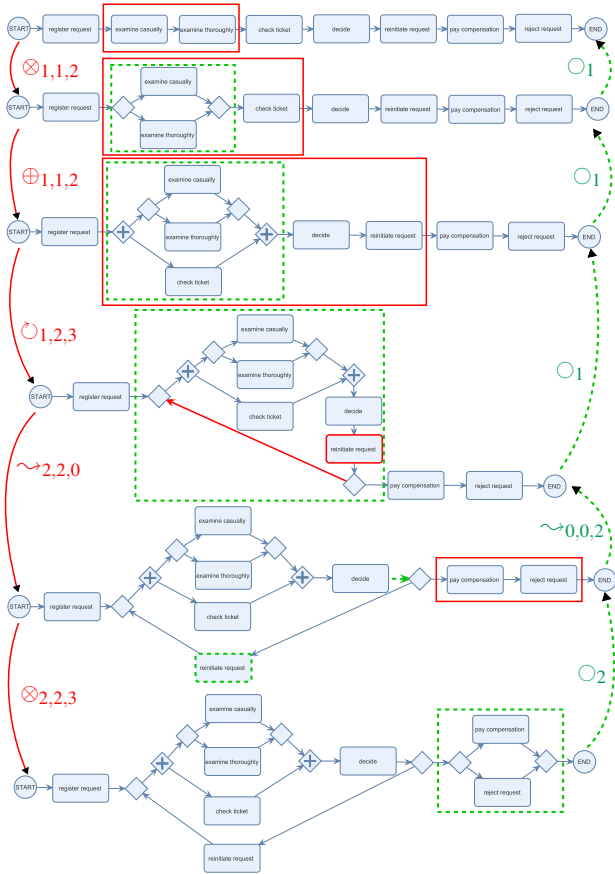


Figure 2: Examples of Operators on a Hierarchical Model

was at). Also, the set B is modified by removing the two gateways belonging to the atomic graph to be simplified:

$$\odot : (\mathcal{H}, \mathcal{A}) \rightarrow \mathcal{H}$$

$$\odot((\mathcal{A}, \mathcal{G}, B), \alpha) = (\mathcal{A} \setminus \{\alpha\}, (\mathcal{G} \setminus \{\mathfrak{s}_1, \mathfrak{s}_2, \mathfrak{s}_3\}) \cup \{\mathfrak{s}'_3\}, B \setminus \{f, l\})$$

where

$$\alpha = (f, l, \mathfrak{s}_1, \mathfrak{s}_2) \wedge$$

$$\exists i \in \mathbb{N} \mid (i, \alpha) \in \mathfrak{s}_3 \wedge$$

$$\mathfrak{s}'_3 = \text{ins}(\mathfrak{s}_1, 0, \infty, \text{ins}(\mathfrak{s}_2, 0, \infty, \text{rmv}(\mathfrak{s}_3, i, i), i), i)$$

i.e. the atomic graph α to be simplified is located at index i in its parent sequence $\mathfrak{s}_3$.

Figure 2 shows an example of a business process model and the operators applied on it. The left side of the figure shows three complexify operations, one move operation, and another complexify operation being applied on an initially strictly sequential model, evolving it into a more complex model from the top of the figure to the bottom. The right side of the figure shows the inverse operations being applied on the model from bottom to top (one simplify, followed by one move and another three simplify operations). The complexify operator symbols on the left side of the picture have three indices which specify the parameters k, l, m , i.e. the indices in a sequence of atomic graphs denoting the part of the sequence to be complexified. For instance, the operation $\oplus_{1,1,2}$ performed on the main process sequence creates a graph at index 1 with two parallel branches, the first branch containing the graphs from index 1 (inclusive) to index 1 (inclusive) and the second containing the graphs from index 1 (exclusive) to index 2 (inclusive). The simplify operator symbols have one index which denotes the position

Algorithm 1: Evolution Strategy

Data: $\log$

Result: bestModel

```

1 begin
2    $\text{initialModel} \leftarrow \text{createSequentialModel}(\log)$ ;
3    $\text{pop} \leftarrow (\text{initialModel})$ ;
4   for  $i=2$  to  $\mu$  do  $\text{pop} \leftarrow \text{pop} \circ \text{mutate}(\text{initialModel})$ ;
5   while not terminated do
6      $\text{newPop} \leftarrow \text{pop}$ ;
7     for  $i=1$  to  $\lambda$  do
8        $\text{parentModel} \leftarrow \text{selectUniform}(\text{pop})$ ;
9        $\text{newPop} \leftarrow \text{newPop} \circ \text{mutate}(\text{parentModel})$ ;
10    end
11     $\text{sortDescending}(\text{newPop})$ ;
12     $\text{pop} \leftarrow \text{newPop}_0 \dots \text{newPop}_{\mu-1}$ ;
13     $\text{bestModel} \leftarrow \text{pop}_0$ ;
14  end
15 end

```

of the atomic graph in a sequence to be simplified, e.g. $\odot_1$ simplifies the atomic graph at index 1 in a specified sequence of atomic graphs. Similarly, the indices at the move operator specify the part of a sequence to be moved and the destination. For instance, $\sim_{2,2,0}$ moves the range of atomic graphs from index 2 (inclusive) to index 2 (inclusive) of a specified source sequence to index 0 in a specified destination sequence. For visual purposes, the parameters specifying the affected sequences are not shown in Figure 2. Instead, the parts affected by the operations on the left side are highlighted in the picture (red, thick), as well as the parts affected by the inverse operations on the right side (green, dashed). Using a sequence of the above described operations, any process model conforming to the definition of the hierarchical perspective can be transformed to any other model conforming to the same definition.

4. EVOLUTION STRATEGY

For mining process models from a given event log we use a $(\mu + \lambda)$ -ES (Algorithm 1), where the individuals are process models represented by their control-flow model (used for determining fitness, i.e. a process model is fit if it conforms well to the given event log) and their hierarchical view (for performing operations on the model). We start the evolution strategy by creating a sequential model that contains all distinct activities occurring in the event log (ordered by their first appearance), and mutating that model to create an initial population of μ process models. Throughout the optimization, we use a uniform random selection for selecting parents and a best selection as survivor selection. For the experiments carried out in this paper, we use a population size of $\mu = 1000$ individuals and generate $\lambda = 5000$ child models in each generation.

4.1 Mutation

In order to create a mutation of a process model we copy it and modify this copy by randomly applying some of the operators *move*, *complexify* and *simplify*. With a finite concatenation of these three operators any valid hierarchical model can be reached from any other valid hierarchical model, thus permitting the algorithm to find the best fitting model. The parameters for each operation are also randomized within boundaries ensuring model validity (e.g. which atomic graphs are moved to which sequence by $\sim$, the possible destination sequences being limited to ones described in Section 3). In addition to avoiding the creation of invalid models the algorithm to some degree anticipates operations which have no

effect on the model behaviour. For instance, in cases where the model is strictly sequential, we remove $\circ$ from the set of possible operators. When moving parts of the model we ensure they are moved to a different position, and after moving the last atomic graph from a complex construct with a $\sim$ operation, the construct is automatically removed (simplified).

While there is no self-adaptation of the step-size or mutation strength in our implementation, we perform a random concatenation of multiple of the above mentioned operators in one mutation (to allow for a sufficient amount of change in process model behaviour to be introduced).

4.2 Diversity

In evolutionary algorithms, a balance of exploration and convergence is important for overcoming local optima and refining solution candidates' fitness. A diverse population is desired to ensure good coverage of the search space, thus reducing the likelihood of missing the part of the search space in which the global optimum resides. If there is no individual near the global optimum, but the whole population has converged to a local optimum, the benefits of population-based optimization are eliminated. Therefore, a diversity measure – indicating how similar the individuals of a population are – can be used to monitor the optimization and take countermeasures if the optimization has converged prematurely.

One option to measure population diversity for individuals with a binary fixed-length genotype is to compute the sum of hamming distances between all individuals in a population (pair-wise). Since this computation has a quadratic runtime-complexity (with regards to the population size) it is not suitable for large populations, such as ≥ 1000 individuals). We therefore measure diversity by computing the sum of absolute distances from each individual to the “mean individual” of the population. We consider a population p to be a vector of individuals, and each individual p_i as a vector of zeroes and ones, i.e. its genotype in shape of a bit string (in our context of process discovery, an individual is a process model and its bit string representation reflects a direct-neighbour relationship between activities in the process model). The mean individual $\bar{p}$ of a population with n individuals is defined as:

$$\bar{p} = \frac{\sum_{i=1}^n p_i}{n}$$

Note that the contents of the mean genotype $\bar{p}$ are real-valued, between 0 and 1. We then define the population diversity as the sum of Manhattan distances from each individual to the mean individual:

$$div = \sum_{i=1}^n \|p_i - \bar{p}\|_1$$

In order to draw conclusions from a diversity value, it is useful to know the boundaries for diversity. While the minimum diversity of 0 is reached if all individuals are equal, the maximum diversity depends on the length of the genotype and the size of the population. Maximum diversity across the whole length of the genotype is reached when the diversity is maximal for every single bit of the genotype (if the bits are independent). The population diversity in a single bit j is defined as:

$$div_j = \sum_{i=1}^n |p_{ij} - \bar{p}_j|$$

Let $\#I_j$ denote the number of individuals in population p which have a 1 in bit j . The number of individuals with a 0 in bit j is then $\#0_j = n - \#I_j$. The value of the mean individual at index j is

$\bar{p}_j = \frac{\#I_j}{n}$. The diversity in bit j can then be expressed as a function of the number of ones in bit j :

$$\begin{aligned} div_j(\#I_j) &= \#0_j * \bar{p}_j + \#I_j * (1 - \bar{p}_j) \\ &= (n - \#I_j) * \frac{\#I_j}{n} + \#I_j * (1 - \frac{\#I_j}{n}) \\ &= 2 * \#I_j - 2 * \frac{\#I_j^2}{n} \end{aligned}$$

The first derivative is then:

$$\frac{\Delta div_j(\#I_j)}{\Delta \#I_j} = 2 - 4 * \frac{\#I_j}{n}$$

with its root at

$$\begin{aligned} 0 &= 2 - 4 * \frac{\#I_j}{n} \\ \#I_j &= \frac{n}{2} \end{aligned}$$

The second derivative test yields $-\frac{4}{n} < 0$ indicating a maximum in the diversity of

$$\begin{aligned} maxdiv_j &= div_j(\frac{n}{2}) \\ &= 2 * \frac{n}{2} - 2 * \frac{(\frac{n}{2})^2}{n} \\ &= \frac{n}{2} \end{aligned}$$

This means the diversity in bit j reaches its maximum of $\frac{n}{2}$ if half of the population has a 1 in bit j and the other half has a 0 in bit j . For a population where every one of the n individuals has a genotype of length g , the maximum diversity is:

$$maxdiv = \frac{n}{2} * g$$

4.3 Fitness

The goal of process discovery is to find a process model that best represents a given event log. Hence, we evaluate the fitness of process models by comparing the behaviour they reflect to the behaviour that is evident in the event log. The evolution strategy tries to maximize the coherence of log and model behaviour, i.e. models that better represent the event log are considered fitter. An event log is split into *traces*, where each trace is a sequence of events logged during a single process instance, i.e. the execution of the business process from the start node to the end node. Each logged event represents the execution of a single activity. Consider the event log $L = \{(a,b)^2, (b,a)^2(b,a,c)^4, (a,b,c,c,c)^3\}$ which consists of 11 traces, of which 4 are unique (i.e. the process has been executed in 4 different variants). A process model reflecting this event log should contain the three activities a , b , and c . They should be organized in such a way that the process starts with a parallel construct with a in one branch and b in the other (since the event logs shows that a and b maybe be executed in an interchangeable order). After that parallel construct, a loop construct should follow which allows for executing activity c multiple times (or not at all). The starting point for comparing log and model behaviour is the *directly-follows footprint*, a relation between activities that indicates whether they may directly follow each other in the execution of the process. This relation is extracted from the process model by applying graph traversal algorithms, and is also extracted from the log by analyzing its traces. For instance, in the above mentioned log L , activity a may be followed by b or c or no activity (in one of the traces a is the last activity to be executed, i.e. not followed by another event). Activity b may be followed by

a or c or no activity. Activity c can be followed by either itself or no activity. Once we have extracted the directly-follows footprint of the model and the directly-follows footprint of the log, we compare them in two directions: *recall* indicates how much of the behaviour evident in the log is also found in the model (i.e. how many tuples in the log directly-follows relation are also found in the model directly-follows relation), and *precision* indicates how much of the behaviour exhibited by the model is supported by the event log. In addition to this footprint conformance measure which only compares how events / activities directly follow each other, we measure replay conformance, i.e. how capable the model is of replaying whole traces of the event log. We then define the overall conformance between model m and log l as the average of these two conformance measures (described in more detail in [11]):

$$\text{conf}(m, l) = \frac{\text{conf}_{rp}(m, l) + \text{conf}_{fp}(m, l)}{2}$$

Both footprint and replay conformance measures, and thus also the overall conformance measure, yield values in $[0, 1]$. To avoid large models with overly complex branching behaviour we define a fitness function fit_1 that is based on the conformance of model and log, and penalizes models with many loops, exclusive or parallel constructs:

$$\text{fit}_1(m, l) = \text{conf}(m, l) * 0.99^{\#cs(m)}$$

Here, $\#cs(m)$ is the number of exclusive, parallel or loop constructs in the model m . The factor initially reduces the fitness of a model by around 1% for every such construct, with a decreasing penalty in case of complex models.

When performing experiments using this fitness function, however, the population often prematurely converges to local optima. Therefore, we improve the fitness function to avoid early convergence by making it dependent on two more factors: the size of the problem (i.e. the number of activities present in an event log) and the population diversity:

$$\text{fit}_2(m, l) = \text{conf}(m, l) - \frac{\text{div}}{\text{maxdiv}} * \frac{\#cs(m)}{(\#act(l) + 1)^2}$$

where $\#act$ represents the number of distinct activities occurring in a given event log. Here, the penalty for complex models consists of the number of constructs divided by the size of the directly-follows footprint matrix (a matrix which stores for each combination of two activities whether one follows the other or not, including the relation to “no activity”, i.e. the start/end of the process, hence $\#act(l) + 1$). The factor $\frac{\text{div}}{\text{maxdiv}}$ describes how diverse the current population is relative to the maximum obtainable diversity. It is calculated according to the equations in Section 4.2, when considering the directly-follows footprint matrix of a process model as its genotype. This aims to allow for more complex models when population diversity is low, therefore encouraging exploration, while enforcing simplification when population diversity is very high (the rationale is that allowing for higher complexity extends the accessible search space, thus permitting populations with a higher diversity). In experiments with multiple event logs we have found that this dynamic fitness function is less prone to getting stuck in local optima.

5. EVALUATION

We evaluate the performance of our approach by using eight event logs to mine process models. The resulting models are compared to the results of three other state of the art / related process mining algorithms applied on the same event logs. Since the approaches have a different representation of the resulting models,

Table 1: Conformance results of the different discovery algorithms

Log	Trace Fitness				Precision				Simplicity			
	HM	IM	DGEM	GM	HM	IM	DGEM	GM	HM	IM	DGEM	GM
L ₁	0.679	0.863	1.000	0.868	0.532	0.529	0.550	0.497	86	91	78	94
EX5	0.985	0.935	1.000	1.000	0.495	0.560	0.681	0.480	155	102	81	262
REP	1.000	1.000	1.000	1.000	0.905	0.955	0.955	0.943	72	46	49	70
BE1	0.991	1.000	1.000	0.721	0.838	0.814	0.821	0.707	192	132	130	167
BE2	0.924	0.981	1.000	0.882	0.737	0.594	0.684	0.635	196	156	115	203
BE3	0.822	0.983	1.000	0.981	0.891	0.443	0.805	0.732	178	149	114	222
DF	1.000	0.911	1.000	0.906	0.563	0.559	0.588	0.276	177	136	133	176
FL _A	0.974	1.000	1.000	1.000	0.920	0.695	0.953	0.965	98	62	77	193
AVG	0.922	0.959	1.000	0.920	0.735	0.644	0.755	0.654	144.3	109.3	97.1	173.4

all models are transformed into Petri nets to allow for their comparison. To evaluate the quality of the mined process models we use existing measures defined on Petri nets [15]: *trace fitness* (a measure of recall) and *precision*. The plugin *PNetReplayer* in the toolkit ProM [16] is used to determine these two measures. Furthermore, we determine model simplicity by counting the number of elements (places, transitions, and arcs) in the resulting Petri nets. A low number of elements indicates a simpler, easier to understand, model.

The first of eight event logs used for evaluating the process mining algorithms is defined as $L_1 = \{ (b, a)^4, (a, b, d, e)^5, (b, a, e, d)^4, (b, a, c, a, b, c, b, a, d, e, e, d)^6, (g, g)^2, (f, h)^3, (f, f, h, f, g, h, g, f, h)^8, (g, h, f)^2 \}$. It contains 34 traces of which eight are unique, and eight activities. EX5 and REP are example event logs that come with the toolkit ProM (exercise5 which contains 14 activities, and repairExample which contains 8 activities). The three logs BE1, BE2, and BE3 have been generated by us, using simulation on randomly created complex models with 20 activities (the models were created by performing a sequence of the operators described in Section 3 on initially sequential models with 20 activities). Log DF is extracted from an eHealth process which contains 18 activities [7], and FL_A is an excerpt of a loan application process log with ten activities used in the Business Process Intelligence Challenge 2012 (process “A”).

The three state of the art mining algorithms used for comparison are the heuristic miner (HM) [1], inductive miner (IM) [9] and genetic miner (GM) [2]. HM uses heuristic nets as process representation, and is considered as one of the best process mining algorithms [5]. GM also mines heuristic nets, using a genetic algorithm. IM mines process trees, which are a block-structured representation, similar to the hierarchical model we use. Both HM and IM are deterministic algorithms which require little time to execute, even for big event logs. For GM and our approach (called DGEM) we provide a maximum of two hours optimization time per event log and take the best model obtained after this time.

The performance of the four algorithms on the eight event logs is presented in Table 1. The bottom row shows for each algorithm and conformance metric the average score over all event logs. The best result for each event log and metric is marked bold and green while the worst result is marked with red colour. Our approach (DGEM, using fitness function fit_2) performs well across all event logs, achieving highest scores in recall (trace fitness) while maintaining best or second best scores in precision and simplicity. Due to the use of well structured models both IM and DGEM score much better in simplicity than HM and GM. For the log FL_A algorithm GM found a model with superior recall and precision. This model is, however, roughly three times as big as the one found by DGEM. On average, HM scores very well in precision, where it outperforms DGEM in the three event logs which have been generated by simulating complex models.

In Figure 3 the results of the different mining algorithms, when mining a model from log EX5, are shown in the shape of Petri nets.

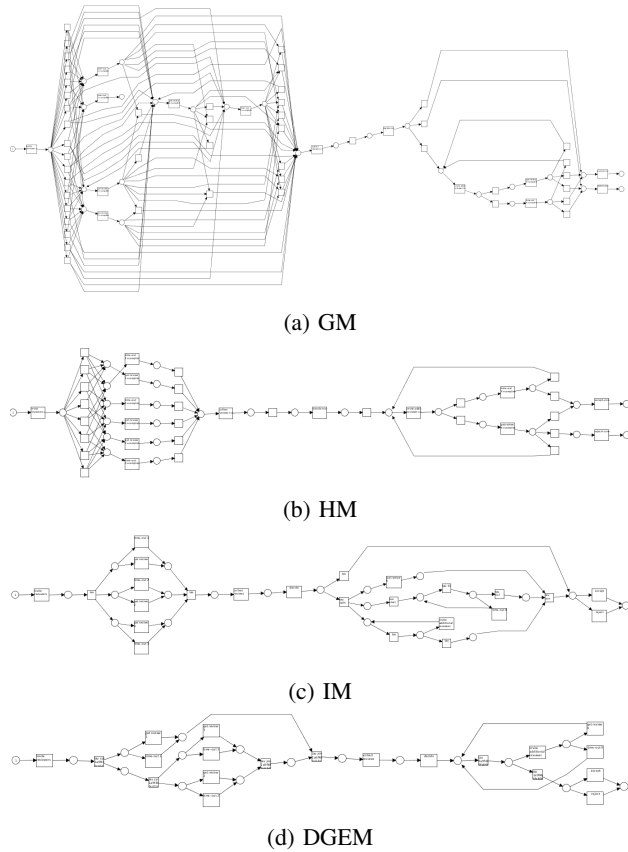


Figure 3: Models mined by the different algorithms for log EX5, a log pertaining to a scientific publication review process

The pictures reflect the conclusions drawn from the previous evaluation regarding the simplicity of process models: IM and DGEM tend to produce results which are much easier to read. Note that in this case, however, the model mined by IM only reflects 93.5% of the behaviour in log EX5 whereas the other algorithms' results have a trace fitness close or equal to a 100%.

Figure 4 shows the results of a performance comparison between GM (thin lines) and DGEM (thick lines) carried out using the event log DF. The chart shows how trace fitness, precision, and simplicity change over the course of the optimization. Note that DGEM relatively quickly finds a model with good trace fitness (reaches perfect trace fitness of 1.0 after 12 seconds) at the cost of precision and simplicity. Only after no more improvements in trace fitness can be made, the algorithm improves precision and simplicity. This is likely due to the higher emphasis of recall in the fitness function (replay and footprint) of DGEM. After one minute of optimization, no more improvements are found. In contrast, the genetic miner takes 30 minutes to find a model with close to perfect trace fitness, while simplicity and precision – although slightly improving over time – do not reach the results of DGEM, even after 2 hours of optimization.

Figure 5 shows the progress of two runs of our evolution strategy on the event log DF, each with a different fitness function of the ones defined in Section 4.3 (fit_1 on the left, and the diversity-influenced fitness function fit_2 on the right). In the pictures, the fitness (best, average, and worst individual) and the population diversity are plotted over the number of generations. In Figure 5, the fitness is monotonically increasing over the course of the optimization while the diversity is the fluctuating line, being high in the

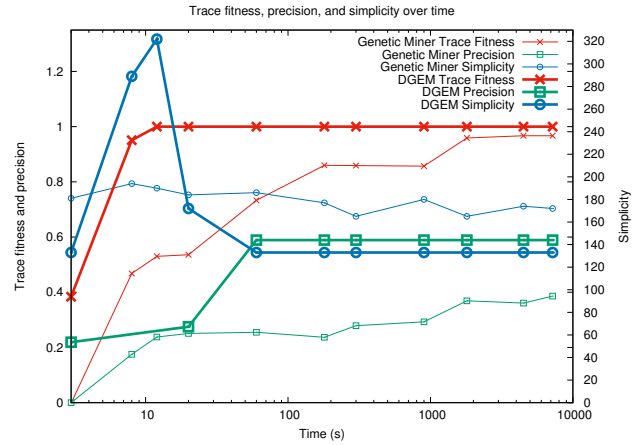


Figure 4: Performance of DGEM and GM

beginning and converging to zero once the optimum is reached. As can be seen, the run with the normal fitness function on the left side shows several plateaus in the fitness value, e.g. from generation 200 to 550 there has been no fitness improvement. One can also see that the fitness improvements around generation 550 are preceded by a build-up of population diversity (starting around generation 430). Here, the optimum is reached after 700 generations. In contrast, when using the diversity-dependent fitness function (the picture on the right side), the diversity does not flatten out at zero until the optimum is reached. Also, the progress of the fitness is different: there is a steady increase to the optimum, which is already reached at generation 180. While the results are hard to compare (since different fitness functions are used in both figures), the resulting process models were the same in both cases. We observed this kind of pattern repeatedly throughout the set of eight event logs we used for evaluation. In the more complex cases (BE1-3), the results of the optimizations runs with diversity-dependent fitness function consistently outscored the results obtained using the standard fitness function (within a time frame of 2h).

6. RELATED WORK

Several techniques have been developed for the discovery of Petri nets from event logs [3]. An approach that also uses computational intelligence and similar notions of conformance for evaluating process models is presented in [2], where the authors use a genetic algorithm to mine Petri nets (without any further structure than given by the basic Petri net elements). In [17] a two step approach mines first a state transition system and then transforms it into Petri nets. The approach does not consider noise, but as in our approach it allows for user guided discovery, by letting the user choose from different abstractions to balance underfitting and overfitting.

A type of model that is structured in a way similar to the hierarchical model presented in Section 2 is defined by process structure trees [6]. Whereas process structure trees are model representations of business processes themselves which can be transformed into other representations such as Petri nets or business process models, our hierarchical model is a view on top of an existing business process model. In [4] an approach for mining process structure trees is presented that uses genetic algorithms. Compared to the operators defined in Section 3 the mutation operators used in [4] are of a lower level nature. For instance, the $\leadsto$ operator we use to move a part of a sequence of atomic graphs would be expressed by several remove and add operations in [4]. In contrast to our approach the authors allow for duplicate activities which can lead to unrealistic, overfitting models (e.g. a big model consisting of a single

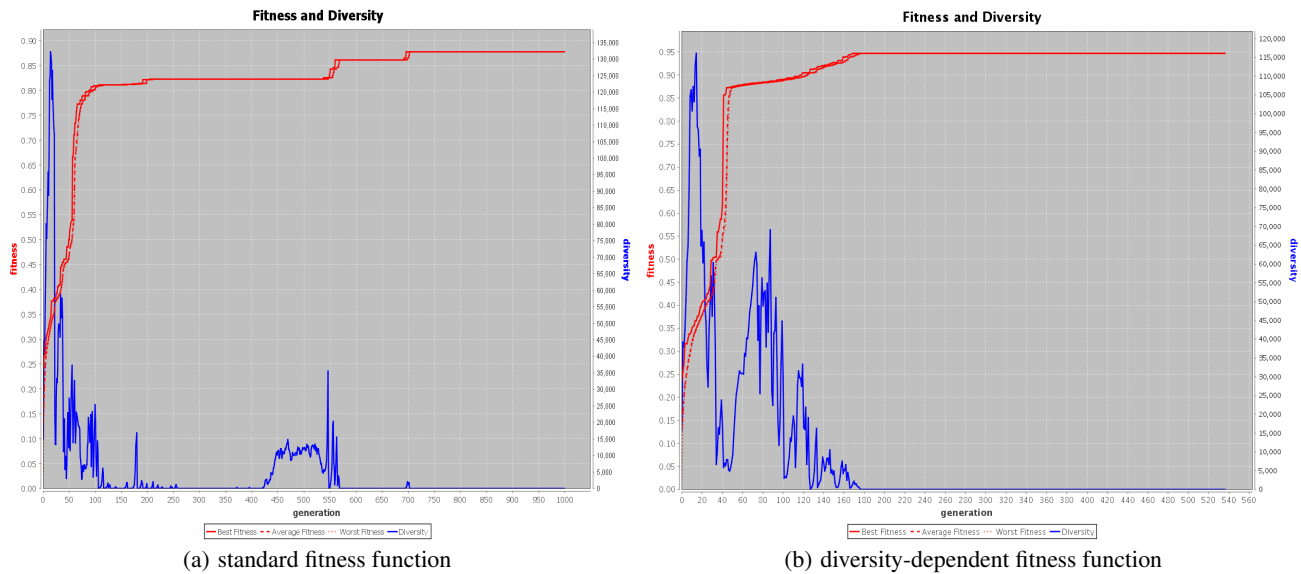


Figure 5: Comparison of evolution progress when using different fitness functions

exclusive choice between many branches, each representing a trace in the log). As in our approach, fitness is evaluated as a weighted average of several conformance measures, with the weights being defined by the user to allow for guided process discovery. However the precision measure used in [4] does not detect underfitting very well (one of the mined examples with a perfect precision score exhibits loop behaviour which is not present in the original model used to produce the log, thus allowing for more behaviour than in the log).

In [8] an approach for diversity guided evolutionary algorithms is proposed which also uses the distance to the “average” individual for measuring diversity. This approach controls the optimization by switching between two discrete modes (exploitation, i.e. recombination, and exploration, i.e. mutation), based on the absolute population diversity exceeding some pre-defined thresholds. Our approach, in contrast, continuously adapts the fitness function by determining how far the current diversity is from the maximum obtainable diversity.

7. CONCLUSION

We have presented an evolutionary computation based approach to mine BPMN based hierarchically structured process models from business process event logs. Our approach is able to find process models which are small and easy-to-understand when compared to the results of existing process mining approaches, and yet maintain a high accuracy in reflecting the behaviour of the event log. By defining a diversity-adapted fitness function, the optimization avoids early convergence to local optima.

Our approach may be further improved by examining which sequences of operators ($\leadsto$, $\oplus$, $\otimes$, $\odot$, $\circ$) lead to successful individuals, and using this information to create self-adaptive mutations. Furthermore, an extensive evaluation of all parameters such as population size, number of mutations, methods for parent and survivor selection, etc. is expected to provide a set-up which yields better results.

Whether the concept of using relative diversity to guide the optimization may be applied in other use cases of evolutionary computation is subject to further research. For instance, in fitness-proportionate selection, relative diversity could be used for scaling fitness values (e.g. as alternative to sigma-scaling).

8. REFERENCES

- [1] A. J. M. M. Weijters and J. T. S. Ribeiro, *Flexible Heuristics Miner (FHM)*. IEEE Symposium on Computational Intelligence and Data Mining. 2011.
- [2] A. K. Alves De Medeiros, A. J. M. M. Weijters, and W.M.P. van der Aalst, *Genetic process mining: an experimental evaluation*. Data Mining and Knowledge Discovery. 14(2): 245-304, 2007.
- [3] B. F. Dongen, A. K. Alves de Medeiros, and L. Wen, *Process Mining: Overview and Outlook of Petri Net Discovery Algorithms*. Transactions on Petri Nets and Other Models of Concurrency II: 225-242, 2009.
- [4] J. C. A. M. Buijs, B. F. Van Dongen, and W. M. P. van der Aalst, *A Genetic Algorithm for Discovering Process Trees*. 2012 IEEE Congress on Evolutionary Computation, 2012.
- [5] J. De Weerd, M. De Backer, J. Vanthienen, B. Baesens. *A multi-dimensional quality assessment of state-of-the-art process discovery algorithms using real-life event logs*. Information Systems 37(7): 654-676, 2012.
- [6] J. Vanhatalo, H. Voelzer, J. Koehler. *The Refined Process Structure Tree*. BPM 2008 Proceedings of the 6th International Conference on Business Process Management, 2008.
- [7] M. Galushka, W. Gilani, *DrugFusion - Retrieval Knowledge Management for Prediction of Adverse Drug Events*. 17th International Conference on Business Information Systems, 2014.
- [8] R. K. Ursem, *Diversity-Guided Evolutionary Algorithms*. Parallel Problem Solving from Nature - PPSN VII, 2002.
- [9] S. J. J. Leemans, D. Fahland, W. M. P. van der Aalst, *Discovering Block-Structured Process Models from Event Logs Containing Infrequent Behaviour*. Business Process Management Workshops, 2013.
- [10] T. Allweyer. *BPMN 2.0 – Introduction to the Standard for Business Process Modeling*. BoD, 2010.
- [11] T. Molka, D. Redlich, M. Drobek, A. Caetano, X.-J. Zeng, and W. Gilani, *Conformance Checking for BPMN-Based Process Models*. Proceedings of the 29th Annual ACM Symposium on Applied Computing, 2014.
- [12] W. M. P. van der Aalst. *Process mining: discovering and improving Spaghetti and Lasagna processes*. In Proceedings of the IEEE Symposium on Computational Intelligence and Data Mining (editors: N. Chawla, I. King and A. Sperduti): 13-20, 2011.
- [13] W. M. P. van der Aalst, *Process Mining: Discovery, Conformance and Enhancement of Business Processes*. Springer, 2011.
- [14] W. M. P. van der Aalst et al, *Process Mining Manifesto*. BPM 2011 International Workshops, 2011.
- [15] W. M. P. van der Aalst, A. Adriansyah, B. F. van Dongen, *Replaying History on Process Models for Conformance Checking and Performance Analysis*. WIREs Data Mining and Knowledge Discovery, 2(2): 182-192, 2012.
- [16] W. M. P. van der Aalst, B. F. van Dongen, *ProM : The Process Mining Toolkit*. Industrial Engineering, 489: 1-4, 2009.
- [17] W. M. P. van der Aalst, V. Rubin, H. M. W. Verbeek, B. F. van Dongen, E. Kindler, C. W. Günther. *Process mining: a two-step approach to balance between underfitting and overfitting*. Software and System Modeling, 9: 87-111, 2010.